

05. Travel Agent - Servizio Agent

6. Implementazione del Servizio Agent

Crea la cartella services e il file agent_service.py

A questo punto la struttura del progetto dovrebbe essere completa

```
├─ README.md
├─ poetry.lock
├─ pyproject.toml
├─ tests
│   └─ __init__.py
└─ travel_agent_api
    ├── __init__.py
    ├── main.py
    ├── routes
    │   └─ chat_router.py
    ├── services
    │   └─ agent_service.py
    └─ tools
        ├── chain_historical_expert.py
        ├── chain_travel_plan.py
        ├── flights_finder.py
        └─ hotels_finder.py
```

Il servizio Agent è il cuore dell'applicazione orchestrando tutti i tool:

```
from datetime import datetime
from langchain_openai import ChatOpenAI
from langgraph.prebuilt import create_react_agent
# Tools
from travel_agent_api.tools.flights_finder import flights_finder
from travel_agent_api.tools.hotels_finder import hotels_finder
from travel_agent_api.tools.chain_historical_expert import chain_historical_expert
from travel_agent_api.tools.chain_travel_plan import chain_travel_plan

FLIGHTS_OUTPUT = """
format: markdown
  ## Miglior Opzione

  ### Andata:
  - Compagnia aerea: Ryanair
  - Data di partenza: 2024-12-13
  - Ora di partenza: 10:00
  - Durata del volo: 1h 30m

  ### Ritorno:

  - Compagnia aerea: Ryanair
  - Data di ritorno: 2024-12-19
  - Ora di ritorno: 14:30
  - Durata del volo: 1h 30m

  Inserisci il link di Google per la prenotazione se possibile.
```

```
#### Altri opzioni disponibili:
```

- Compagnia aerea: Ryanair
- Data di partenza: 2024-12-13
- Ora di partenza: 10:00
- Durata del volo: 1h 30m
- Compagnia aerea: Ryanair
- Data di ritorno: 2024-12-19
- Ora di ritorno: 14:30
- Durata del volo: 1h 30m

```
...
```

```
####
```

```
HOTELS_OUTPUT = ""
```

```
format: markdown
```

```
#### Hotel 1
```

```
Inserisci la foto dell'hotel se disponibile.
```

```
*Descrizione:* Camere e suite eleganti, a volte con vista sulla città, in hotel esclusivo con piscina panoramica e spa.
```

```
*Prezzo per notte:* €296 (prima delle tasse e spese: €260)
```

```
*Prezzo totale:* €2,660 (prima delle tasse e spese: €2,336)
```

```
*Check-in:* 15:00, Check-out: 12:00
```

```
*Valutazione complessiva:* 4.5 su 5
```

```
*Servizi Inclusi:* Spa, Piscina, Parcheggio gratuito
```

```
#### Hotel 2
```

```
Inserisci la foto dell'hotel se disponibile.
```

```
Descrizione: Hotel in stile Liberty con alloggi arredati in maniera artistica, ristorante elegante, bar e spa.
```

```
*Prezzo per notte:* €380 (prima delle tasse e spese: €333)
```

```
*Prezzo totale:* €3,418 (prima delle tasse e spese: €3,000)
```

```
*Check-in:* 15:00, Check-out: 12:00
```

```
*Valutazione complessiva:* 4.5 su 5
```

```
*Servizi Inclusi:* Spa, Piscina, Parcheggio gratuito
```

```
####
```

```
TRAVEL_PLAN_OUTPUT = ""
```

```
format: markdown
```

```
### Itinerario:
```

```
### Giorno 1 - 2024-12-13:
```

```
*Mattina:* Descrizione dell'attivita' da svolgere la mattina
```

```
*Pomeriggio:* Descrizione dell'attivita' da svolgere il pomeriggio
```

```
*Sera:* Descrizione dell'attivita' da svolgere la sera
```

```
### Giorno 2 - 2024-12-14:
```

```
*Mattina:* Descrizione dell'attivita' da svolgere la mattina
```

```

    *Pomeriggio:* Descrizione dell'attivita' da svolgere il pomeriggio

    *Sera:* Descrizione dell'attivita' da svolgere la sera

    ...

"""

class Agent:
    def __init__(self):
        self.current_datetime = datetime.now()
        self.model = ChatOpenAI(model_name="gpt-4o")
        self.tools = [
            chain_historical_expert,
            flights_finder,
            hotels_finder,
            chain_travel_plan,
        ]
        self.agent_executor = create_react_agent(self.model, self.tools)
        pass

    def run(self, messages : list):
        SYSTEM_PROMPT = f"""
        Sei un travel planner. Il tuo compito e' organizzare il viaggio per l'utente.
        Aggiungi delle emojis per rendere il tuo output piu' interessante.
        La data di oggi e' {self.current_datetime}

        Usa le seguenti istruzioni per creare un output:

        Esempio Ouput Voli:

        {FLIGHTS_OUTPUT}

        Esempio Output Hotel:

        {HOTELS_OUTPUT}

        Esempio di Output Viaggio:

        {TRAVEL_PLAN_OUTPUT}

        """

        conversation_history = [{"role" : "system", "content" : SYSTEM_PROMPT}] + messages
        response = self.agent_executor.invoke({"messages" : conversation_history})
        return response["messages"][1:]

```

Descrizione

Questo codice implementa un agente di viaggio automatizzato che aiuta gli utenti a pianificare i loro viaggi. L'agente è progettato per gestire tre aspetti principali della pianificazione: ricerca voli, ricerca hotel e creazione di itinerari di viaggio.

La classe `Agent` è il componente centrale che orchestra tutte le funzionalità. Quando viene inizializzata, configura:

- Un timestamp corrente per tracciare la data e l'ora

- Un modello di linguaggio GPT-4 tramite l'interfaccia ChatOpenAI
- Una serie di strumenti specializzati per le diverse funzioni di pianificazione viaggio
- Un executor che combina il modello e gli strumenti usando l'architettura ReAct

Il metodo `run()` è il punto di ingresso principale che:

1. Costruisce un prompt di sistema dettagliato che include esempi di formattazione per voli, hotel e itinerari
2. Combina questo prompt con la cronologia dei messaggi dell'utente
3. Invoca l'agente per generare una risposta appropriata

L'output dell'agente è strutturato in modo coerente seguendo dei template predefiniti:

- Per i voli: mostra l'opzione migliore con dettagli di andata e ritorno, seguita da alternative
- Per gli hotel: presenta le opzioni con descrizioni dettagliate, prezzi, servizi e valutazioni
- Per gli itinerari: organizza le attività giorno per giorno, suddivise in mattina, pomeriggio e sera

Una caratteristica interessante è l'uso di emoji per rendere l'output più coinvolgente e facile da leggere. Inoltre, l'agente mantiene la formattazione in markdown per una presentazione pulita e strutturata delle informazioni.

L'agente utilizza diversi strumenti specializzati importati da `travel_agent_api.tools`:

- `flights_finder` per la ricerca dei voli
- `hotels_finder` per la ricerca degli hotel
- `chain_historical_expert` probabilmente per informazioni storiche sulle destinazioni
- `chain_travel_plan` per la generazione degli itinerari

Questa architettura modulare permette all'agente di essere facilmente esteso con nuove funzionalità o modificato per gestire diverse esigenze di pianificazione viaggio.